MC-CIM: A Reconfigurable Computation-In-Memory For Efficient Stereo Matching Cost Computation

Zhiheng Yue[1], Yabing Wang[1], Leibo Liu[1], Shaojun Wei[1], Shuoyi Yin[1*]
[1]Tsinghua University, Beijing, China
*Corresponding Author: yinsy@tsinghua.edu.cn

ABSTRACT

This paper proposes the design of a computation-in-memory for stereo matching cost computation. The matching cost computation incurs large energy and latency overhead because of frequent memory access. To overcome previous design limitations, this work, named MC-CIM, performs matching cost computation without incurring memory access and introduces several key features. (1) Lightweight balanced computing unit is integrated within cell array to reduce memory access and improve system throughput. (2) Self-optimized circuit design enables to alter arithmetic operation for matching algorithm in various scenario. (3) Flexible data mapping method and reconfigurable digital peripheral explore maximum parallelism on different algorithm and bit-precision. The proposed design is implemented in 28nm technology and achieves average performance of 277 TOPs/W.

KEYWORDS

Computation-In-Memory, 3D perception, Stereo Vision, Matching Cost, Reconfigurable Design

1 INTRODUCTION

Real-time stereo vision for depth computation is increasingly popular in fields like robotics, self-driving vehicles. Stereo matching estimates the depth information based on distance between matching pixels of pair images. Matching cost computation iterates exhaustively for entire image to figure out corresponding matching pixel. A 50 FPS HD 1280×720 stereo camera generates massive image data (700Mb/s). Associated with higher frame rate and image resolution, matching cost computation imposes high memory bandwidth (~1Tb/s) and computing throughput (> 470 GOPs) requirement. Besides, basic function adopted by matching algorithm varies on different environments, like light resource, item texture. Therefore, the hardware design is required to enhance throughput in various scenario and addresses critical memory challenge.

Digital implementation thus focus on accelerating computing by exploring data parallelism. Massive parallel unit and rotating FIFO are employed to realize real-time stereo matching[1]. Previous proposal also takes the benefit of sparse matching and prediction technique is utilized to skip unnecessary computation[2]. But all digital implementation have fundamental drawbacks that consume large proportion of energy on moving data between PE and buffer.

Nowadays, Computation In Memory (CIM) has already became a prominent method to resolve memory bottleneck by performing in-situ computing. And SRAM based CIM outperforms in stable data endurance and develops with newest technology. However, it is challenging to apply 'CIM method' directly in matching cost computation because of several limitations.

First, arithmetic operation of matching cost is conflicted with the basic computing paradigm of existed SRAM-CIM. Imaging SRAM read operation, BL is charged (discharged) only when WL is activated and stored weight equals to +1 (-1), which is essentially a bit-wise multiplication. When multiple WL are activated at same time, the current of each bitwise computing is accumulated based on the Kirchhoff laws. Therefore, the fundamental computing paradigm of CIM is parallel multiplication and accumulation (MAC). However, matching cost computation, contains operation choice such as XAC and SAD, can hardly take the benefit of CIM. Large design overhead is incurred for other arithmetic computation. For example, a 12T SRAM cim is implemented to perform XAC for binary network with high area overhead[3]. Split-6T is utilized for XAC computing but requires reference circuit for compensation[4].

Second, circuit scheme is fixed in previous CIM design, which is customized for specific application. As one type of operation, like MAC, dominates entire computing process, it is not necessary to activate a secondary function within CIM array. But arithmetic operation of matching cost algorithm varies in different cases. For example, simple and low latency local matching algorithm is enough for non-occlusion and clear boundary cases. Global matching method is preferred in accuracy-driven situation. And basic computation paradigm is different from each other. Therefore, a desired CIM macro is supposed to adapt its computing scheme at runtime to operation requirement of algorithm.

Third, data mapping decides the computing flow and parallelism. Based on CIM macro architecture, activation data is shared by multiple weight and an ideal mapping method figures out maximum possible parallelism. For application like CNN/DNN, the array space for each data dimension is concrete since data reuse pattern is identical even in different layer. But for flexible computing paradigm, resource allocation is not determined. To fully leverage parallelism benefits of CIM, a configurable data mapping method is required to be suitable for different data precision and operated function. Reconfigurable digital unit is also supposed to merge partial result precisely. Otherwise, the accumulation result is incorrect.

To overcome the limitation of previous in memory computing works, we demonstrate an architecture that perform matching cost computation within SRAM array. To the best of our knowledge,

DAC '22, July 10–14, 2022, San Francisco, CA, USA

ACM ISBN 978-1-4503-9142-9/22/07…$15.00
https://doi.org/10.1145/3489517.3530477

this is the first SRAM-CIM designed for matching cost computation and making the following contribution.

- Symmetric XAC computing unit reduces memory access and enlarges readout margin.
- Self-optimized circuit scheme dynamically adjusts cost function to meet algorithm requirement in different scenario.
- Reconfigurable mapping method and programmable digital peripheral explore maximum computing parallelism in different matching cost algorithm and data precision.

The remainder of this article is organized as follows. Section 2 introduces the background of stereo matching and specific matching cost function. Section 3 outlines the structure of the proposed macro. Section 4 presents the configurable mapping method. Section 5 describes the performance and measurement results. Section 6 concludes this article.

2 BACKGROUND

2.1 Matching Cost Computing Challenge

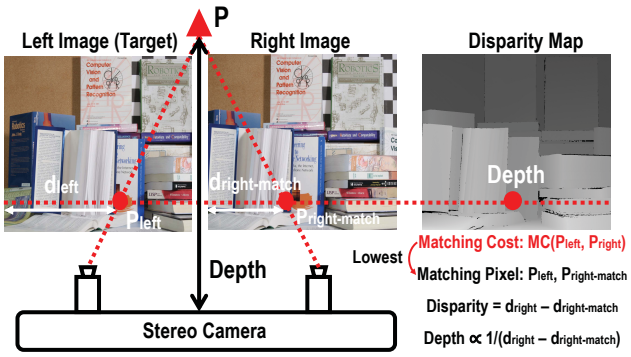


Figure 1: Disparity Map Generation and Epipolar Constraint

Stereo matching acquires depth information based on the distance of two matching pixels. And matching pixels is defined by two pixels with lowest matching cost, like P_{left} and $P_{right-match}$ in Fig. 1. Variable cost function is adopted by matching cost computation based on specific environment. The depth generation process iterates for all pixels in target (left) image. Therefore, the workload equals to W × H × D, where H × W is the image height × width and D represents the size of search region in right image. Fortunately, the epipolar constraint narrows down the range to a horizontal line (dotted line in right image) instead of entire image space.

However, the number of candidates still reach up to 128, matching cost operation for single HD image could exceed 9.4G. Considering high frame rate of real time application, system has to employ large scale parallel computing unit to meet the throughput requirement of algorithm. But vector-wise computing leads to heavy memory traffic. This problem arises in any implementation that separates memory and computation. Frequent memory access limits the computing latency and system energy efficiency. Moreover, customized engine can hardly fit variable cost function, whose basic arithmetic operation is different from each other. Typical cost function will be discussed in section 2.2.

2.2 Cost Function

The Absolute Differences (AD) algorithm (Eq. (1)) compares the differences of intensity between the pixel in the left image and candidate pixels with different disparity locations. The Sum of Absolute differences (SAD) (Eq. (2)) can be viewed as an enhanced version of the AD algorithm as it involves values of pixels around each interest pixel in cost calculation. Both of AD and SAD algorithm are relatively simple for real time application.

$$C_{AD}(x,y,d) = |I_L(x,y) - I_R(x-d,y)| \tag{1}$$

$$C_{SAD}(x,y,d) = \frac{1}{n} \sum_{(a,b)\subset N(x,y)} C_{AD}(a,b,d) \tag{2}$$

$N(x,y)$ is pixel within search window, d is disparity value.

But the performance of AD/SAD degrades in the presence of occlusion or textureless regions. Census Transform (CT) (Eq. (3), Eq. (4)) is proposed to solve this problem by employing relative information between neighboring pixels. Census transform contains two steps as Fig. 2 shows: transform and cost computing.

$$C_{CT}(x,y,d) = ||(CT_l(a,b) - CTr(a-d,b))||_{HD} \tag{3}$$

$$CT(a,b) = Bitstring_{(a,b)\subset N(x,y)}(I(a,b) < I(x_c,y_c)) \tag{4}$$

x_c, y_c represents the center pixel in a census transform window.

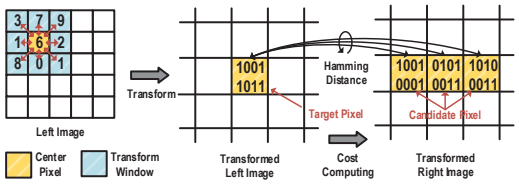


Figure 2: Census Transform

In transform step, surrounding pixel compare with the center pixel. Then all the comparison results are converted into a bit string. Previous work proves that most of comparison is redundant and 98% of transform can be removed by employing circular FIFO[1]. Therefore, this process is computed within digital peripheral and second stage is realized in CIM. In cost computing step, pixel from transformed left image compute hamming distance with candidates pixel in right image.

3 MACRO ARCHITECTURE

To tackle with challenge mentioned in section 1, this work implements a SRAM based CIM, where various arithmetic operations are performed between cell array and shared input. The design includes 64 compute arrays to accelerate computing in parallel. Data reuse is realized by natural scheme of CIM as activation is reused by numerous weights in cell array. The computing unit enables to optimize paradigm at run-time to meet operation requirement of different cost function. And reconfigurable digital unit is implemented to support different bit-precision and computing algorithm.

Fig. 3 presents the overall scheme, which consists of CIM macro, digital peripheral and software configuration. Proposed CIM macro includes two banks (8 compute tile). Each compute tile contains 8 compute array and 1 hierarchal shift/add tree (HSAT). HSAT merges the partial results generated by 8 separated array. The cell array

size equals to 16×64 and includes one computing unit per column. Digital peripheral completes pixel-wise comparison and controls CIM address.

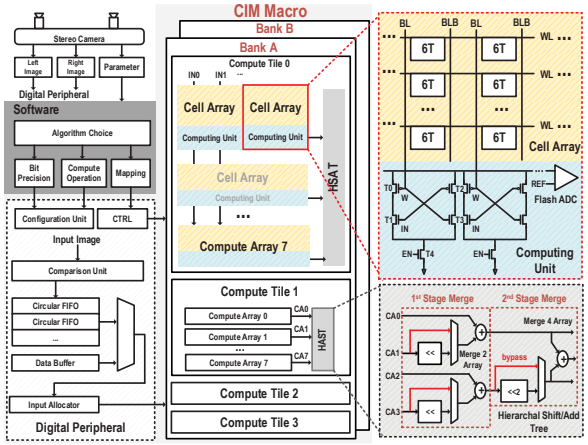


Figure 3: Macro Architecture

3.1 Computing Unit

The computing unit performs column-wise XOR between activated bit cell and input data. XOR is decided as basic arithmetic operation of CIM macro for two reasons. The first one is XOR itself represents difference between two values, which is similar to the goal of matching cost. Moreover, XOR operation could be easily modified into other function, which will be demonstrated in section 3.3. Several XOR/XNOR cim design encounter the issue of sense margin overlapping and need reference circuit to compensate. This is because that computing circuit is non-symmetric, like charge and discharge current is not equalized[4], which shrinks the signal margin and degrades readout accuracy.

Fig. 3 right shows the circuit scheme of computing unit (CU), which is connected with BL of cell array in column-wise. The computing unit has a shape of symmetric x-cross and includes 5 transistors (2 PMOS, 3 NMOS). Data stored in selected bit-cell determines on/off of T0/T3 when WL is activated. Input data controls the gate of T1/T2. T4 acts as the footer to control on/off of CU, which improves power efficiency under low utilization situation.

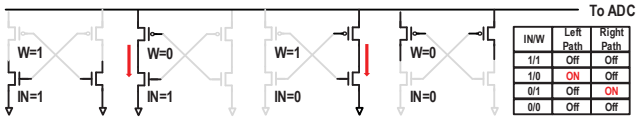


Figure 4: Computing Unit Operation Cases

All possible input combinations of computing unit are shown in Fig. 4. Each bit-wise results are accumulated horizontally and quantized by ADC. As the truth table demonstrates, computing unit performs XOR function when the enable signal is on. Both left and right path cut off and don't contribute current when IN and W equals. Instead, one of paths is turned on when IN and W are different from each other.

Up to 16 bit-wise computation results of CU are accumulated horizontally in the form of current. The analog result will be converted to a partial sum by 4.09b flash ADC in each compute cell. To ensure the correctness of comparison, each voltage ladder is configurable by tuning the voltage off-line. A three stage clocked comparator is implemented to achieve the requirement of resolution and speed.

Same XACV could possibly contain different input and weight combination. The symmetric computing design guarantees that the current contribution by activation and weight is balanced. Therefore, the voltage after accumulation is not coupling with different combination and only linearly related to XACV, which resolves the sense margin overlapping problem mentioned in previous work. The linear computing result relieve the pressure of ADC quantization and improves readout accuracy.

3.2 Hierarchal Shift/Add Tree

Different bit-precision are supported by this design. The weight is split into single bit and saved in cell array, which will be throughly discussed in Section 4. Each compute array is responsible for bit-wise computing. Therefore, the partial result acquired by the compute array is needed to be merged based on its bit significance.

As right bottom of Fig. 3 shows, the shift/adder tree is divided into multiple stages. The first stage will accumulate partial result from two neighboring array and one of results is shifted left by 1 bit. Red line represents the bypass path, in which the partial result will be directly added without shifting. Similarly, the second stage instantiates the first stage component and modifies the shifting value. Three stages are configured in HSAT to support integer power of 2 bit-width like 2, 4, 8.

3.3 Arithmetic Operation of CIM Macro

The matching cost computation algorithm is comprised of multiple basic arithmetic operations. The self-optimize computing unit enables to adjust its basic function for different scenario. Different computing modes supported by the CIM are shown below:

3.3.1 XAC ($\sum_{i=0}^{N} A_i \oplus B_i$). The XOR accumulation (XAC) is basic computing paradigm performed by the CIM Macro. Each column performs XOR operation between input and weight in activated SRAM array. The logical result is accumulated horizontally and converted by flash ADC.

3.3.2 MAC ($\sum_{i=0}^{N} A_i \times B_i$). For binary weight and activation (+1/-1), multiplication equalizes XNOR operation. When input is inverted, XNOR is transformed into XOR and computed by CIM macro.

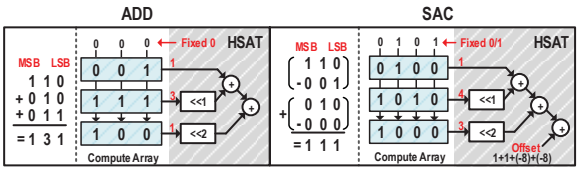


Figure 5: ADD operation and SAC Operation

3.3.3 ADD ($\sum_{i=0}^{N} A_i$). ADD accumulates multiple values stored in same row of CIM array. The input value is fixed to 0, which converts

XAC function into accumulation. For multi-bit ADD operation, one compute array performs bit-wise accumulation. Then the partial result will be post-processed based on their bit-significance by the HSAT as described in previous section.

3.3.4 SAC ($\sum_{i=0}^{N}(A_i - B_i)$). SAC means subtract and accumulation like Fig. 5 shows. In 2's complement representation, subtraction is converted to addition by inverting subtrahend and add 1. The invert is realized by XOR computing unit and fix input data to 1. Then subtraction result is accumulated horizontally. HSAT post-processes the partial results based on each bit-significance and the offset is incurred to acquire the final result. The offset is $N \times (-2^k + 1)$ (Eq. 5), k indicates the bitwidth, N means the number of subtraction group. In the example of Fig. 5 , N equals to 2 and the offset is -14.

$$\begin{aligned}
&\sum_{i=0}^{N}(A_i[k:0] - B_i[k:0]) \\
&= \sum_{i=0}^{N}(A_i[k:0] + (2^{k+1} - 1 - B_i[k:0]) + (1 - 2^{k+1})) \\
&= \sum_{i=0}^{N}(A[k:0] + (2^{k+1} - 1 - B[k:0])) + N \times (1 - 2^{k+1}) \\
&= \sum_{i=0}^{N}(A[k:0] + (\sim B[k:0])) + offset
\end{aligned} \tag{5}$$

3.3.5 SAD ($\sum_{i=0}^{N}|A_i - B_i|$). Sum Absolute Difference (SAD) accumulates absolute value of subtraction. Therefore, the input to CIM are based on comparing result of two pixel. The comparison is similar to transform step of census transform. Each cycle, only one non-overlapping column comparison is performed and pushed into FIFO, which eliminates 85% redundant computing.

One bit SAD is a special case, which simplify the absolute subtraction into an XOR operation. Therefore, the computation scheme is same as XAC. Either subtrahend or minuend is set as input to macro, the other is stored in SRAM array. And the 'Absolute Difference' is completed by XOR computing unit.

4 CONFIGURABLE MAPPING METHOD

This section describes the data mapping for corresponding matching cost algorithm. Census transform and Sum Absolute Difference algorithm are selected as they represent two typical arithmetic operations (XAC and SAD). For different algorithm, the space allocation granularity is different from each other. The configurable mapping method guarantees not only the correctness, but also the computing efficiency, which will be proved in detailed example.

4.1 Census Transform

The basic operation completed by CIM is hamming distance computing (XAC). One transformed pixel from target image is shared with multiple candidates pixel from right image. The parallelism is equivalent to the search window size, which could be up to 128.

Target pixel is regarded as input to CIM macro. Each bit string of right image is located in one row of SRAM array. Target bit string and multiple storage cells in vertical direction perform XOR operation. Then bitwise result is accumulated horizontally to acquire the pop-count. The length of bit string takes half or entire row space and calculation finishes within 5 cycles. It should be noticed that single CIM macro could support 64 compute array performing in-situ computing simultaneously, where stores 64 transformed pixels from different disparity.

As Fig. 6 shows, transformed pixel from left image compute hamming distance with 64 pixels of right image in first iteration. First pixel of transformed right image is located in first row of compute array 0. In second iteration, the search window shift right by one pixel in both image. The pixel 0 is replaced by pixel 64 and other pixels remain in the activating window. To ensure correct activation of pixel 1~64, mapping method allocates pixel 64 on the neighboring row of pixel 0. The column matched mapping is necessary since the input value is shared vertically by all compute array. In second iteration, the compute array 0 and others activate different row due to shifting window. Fine-granularity controlling row addresses of each compute array is possible but induces numerous wiring. To lower the area overhead of row circuitry, a simple 2-to-1 MUX is added to intrinsic row arbiter in each compute array. Only one mask bit is needed to select the address. This design choice considers that only two possible row addresses occur at same cycle.

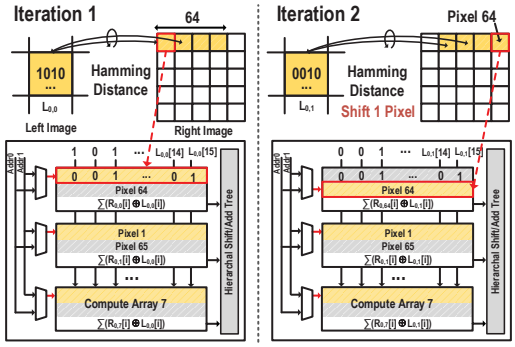


Figure 6: Data Mapping For Census Transform

4.2 Sum Absolute Difference

Data reuse of SAD exists in two dimensions, pixel-wise (multi-bit) and column-wise (one bit). For multi-bit case, image pixel is split into single bit and stored in CIM. The comparison of pair pixel acts as input to macro, which could be shared by all compute array who store same pixel. In 1 bit scenario, the potential computing parallelism raises with shorten bit width. As explained in section 2.1, SAD algorithm accumulates the absolute difference between left and right pixel within a search window. To figure out matching pixel, the search window moves horizontally by one column until all potential disparity are covered. So one column of target image is shared by multiple column in different disparity. Based on this observation, column of left image is conducted as the input of CIM macro and column of right image is stored in each row of compute array.

Fig. 7 illustrates the data mapping for sum absolute difference (SAD). The least significant bit of pixel is stored in compute array 0, compute array 1 saves the second bit, respectively. Each compute array performs bitwise SAD function as described in section 3.3. The

footer controls activation status of computing unit during window shifting. After array ADC converting analog computing current into digital result, HSAT accumulates the bit-wise partial sum based on bit significance.

In one bit scenario, the absolute difference is simplified to logical XOR. The column of right image is stored in each row of compute array. HSAT merges the partial result without shifting. To further improve the utilization, cell array concatenates multiple column pixel once the row space is sufficient.

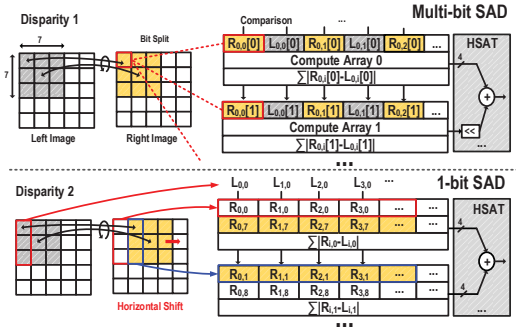


Figure 7: Data Mapping for Sum Absolute Difference

5 EVALUATIONS

This section elaborates on the evaluation of MC-CIM. The computing accuracy of CIM architecture is validated and corresponding power consumption is presented. Several cases are selected to prove the latency reduction of CIM design. Finally, the comparison result with other CIM design is listed.

5.1 Experimental Setup

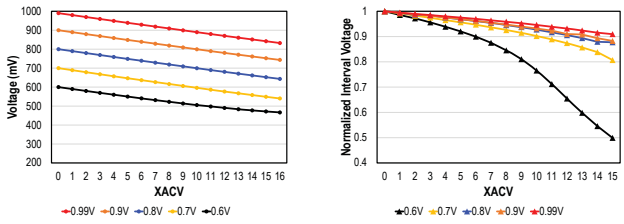
The CIM architecture is implemented in TSMC 28nm process. The SRAM array size is 64Kb (64 compute array). Each compute array consists of 16 row × 64 column cell array, one computing unit per column, 4.09b flash ADC, digital hierarchical shift/adder tree and other peripheral circuit. And the KITTI stereo dataset[5] is selected to estimate the average power and latency.

5.2 Computing Unit Linearity

The computing unit finishes bitwise XOR computing and horizontal accumulation, which contains 5 transistor and incurs 4.95% transistor overhead per column. The relationship between supply voltage and computing linearity is shown in Fig. 8(a). For supply voltage above 0.7V, the XAC voltage could maintain linear from 0 to 16. The linearity degrades much faster at 0.6V supply voltage. Fig. 8(b) presents the interval voltage (normalized to initial interval between XAC 0 and 1) with the scaling of XACV, which could be viewed as the sense margin. By employing symmetric computing design, the interval voltage retains relatively constant for supply voltage above 0.7V, which greatly lowers the quantization error.

5.3 CIM Macro Power Consumption Breakdown

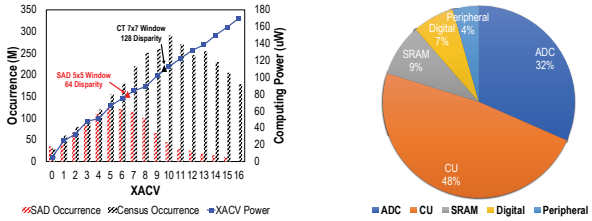
Fig. 9(a) illustrates the power consumption of compute tile under CIM mode (0.9V). The power grows linearly with the increasing



(a) XAC Voltage (b) Interval Voltage

Figure 8: The influence of supply voltage on interval voltage and corresponding MACV voltage is shown : (a) Computing voltage vs XAC value, (b) Interval Voltage vs XAC value.

of XAC value as expected, which is identical to the number of activated current path. To acquire the average computing power of different algorithm, statical analysis is conducted to estimate the average XAC value. The occurrence of XAC value in SAD 5×5 window 64 disparity and CT 7×7 window 128 disparity are shown in Fig. 9(a). In general, SAD algorithm consumes less power than census transform. The reason is throughly explained in section 5.4. Fig. 9(b) demonstrates the power breakdown under computing



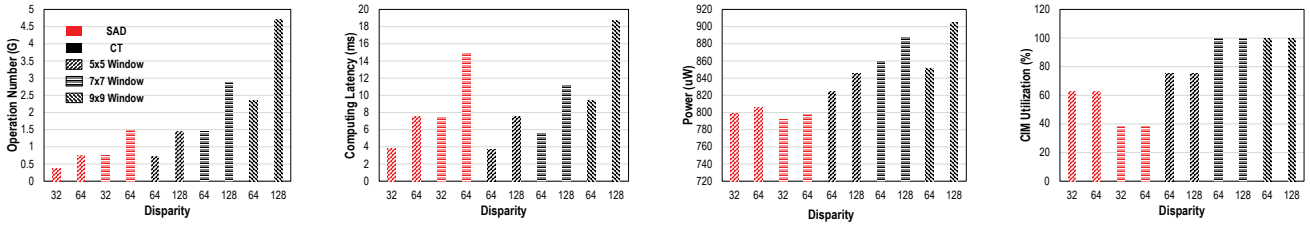
(a) XACV power (b) Power Breakdown

Figure 9: The power consumption of each tile during computing is shown : (a) Power Consumption in Computing Unit, (b) Power Distribution

mode. Near half of the power is consumed by local computing unit as the current flow is larger than other units. Flash ADC accounts for one third power consumption. The other part belongs to sram array, digital unit and other peripheral circuit.

5.4 Arithmetic Operation Computing Latency

Fig. 10(a) compares the operation number of matching computation. The image is selected from KITTI dataset and the resolution is 1240×376. Each bitwise computing like XOR, absolute difference is counted as one operation for simplification. The census transform (w/o transform stage) and SAD contains similar operation when the disparity number is equal. But in general cases, census transform has larger search region and window size. Fig. 10(b) indicates that computing latency grows linearly with the operation number. Experiment proves that this design is suitable for most matching cost algorithm in real-time application. The computing latency for SAD consumes more than 14.9ms since the CIM utilization is limited. The most complex census transform combination consumes longest latency (18.7 ms) because of enormous computing operation.



(a) Algorithm Operation Number (b) Algorithm Computing Latency (c) Power Consumption (d) CIM Utilization

Figure 10: Different window size (3×3, 5×5, 7×7) and disparity number (64, 128) of SAD/CT algorithm are considered: (a) algorithm computing operation, (b) CIM computing latency, (c) Estimated power consumption and (d) CIM activating ratio.

Table 1: Different Computing-In-Memory Comparison

	[6]	[3]	[7]	[8]	This work
Technology	65nm	65nm	65nm	28nm	28nm
Cell	6T	12T	6T + XNOR	0T2R	6T
Array size	4Kb	16Kb	16kb	16Gb[2]	64Kb
Function	XNOR	XNOR	XNOR	XOR	XOR
ADC	Seq SA	Flash ADC	Row ADC	SA	Flash ADC
Latency (ms)	76.88	41.3[1]	19.22	17.72[3]	12.12
TOPs/W	100	139-403	87	4.289	277

[1] 54.21ns clock cycle as described.
[2] Memory scaling to 64kb in performance evaluation.
[3] Assume 8 million 7bit hamming distance operation per cycle at 250MHz.

Power consumption is shown in Fig. 10(c). Statical analysis estimates average XACV of compute array and corresponding power consumption can be predicted by power-XACV curve (Fig. 9(a)). SAD algorithm consumes less power than census transform and has no coupling to disparity number or window size. The reason is each compute array performs bit-wise computing for SAD and the distribution of 0/1 is totally random. The XACV occurrence follows normal distribution for different window size or disparity number in SAD. Instead, array performs pixel-wise computing for census transform. So the XACV is related to ratio of element matching. The power increases with larger window size and disparity since more mismatches are incurred. Fig. 10(d) presents CIM utilization, the ratio of activating computing unit over maximum possible unit. With the assist of 2-to-1 MUX in row circuity, most of census transform algorithm realize full utilization. Otherwise, 50% of compute array remain idle as activating row is out of slide window.

5.5 Comparison

Several computation-in-memory candidates have been selected as they could accomplish XAC operation. And the comparison is shown in Table. 1. KITTI image pair is selected as workload pair. The frequency is set at 250MHz if not mentioned. The estimation of energy efficiency is derived from the experiment of previous work. If not specifically explained, we estimate the computing latency based on the CIM scheme and utilization. The performance proves that the MC-CIM outperforms in energy efficiency and latency. However, the productivity does not scale with the array size linearly, which is due to low utilization of several cases. Besides, the number of activated computing unit is controlled to ensure read out accuracy.

6 CONCLUSION

This work presents a SRAM CIM to improve the efficiency of stereo matching cost computing. First, the macro performs multi-bit arithmetic operation within SRAM without incurring large area overhead. Second, reconfigurable array enables to modify its computing paradigm, which adapt CIM Macro to different matching cost algorithm. Third, flexible data mapping is utilized to fully explore data parallelism based on variable algorithm and bit-precision. This work is using TSMC 28nm CMOS process to confirm the efficiency of the proposed concepts. The experiment of deploying algorithm are conducted and MC-CIM outperforms previous work in latency and energy efficiency. The SRAM-CIM achieved average energy efficiency of 277 TOPs/W for multiple matching algorithm.

ACKNOWLEDGMENTS

This work was supported by the NSFC (62125403 and U19B2041), National Key RD Project (2018YFB2202600), the Beijing ST Project (Z191100007519016) and Beijing Innovation Center for Future Chips.

REFERENCES

[1] Z. Li, Q. Dong, M. Saligane, B. Kempke, L. Gong, Z. Zhang, R. Dreslinski, D. Sylvester, D. Blaauw, and H. S. Kim. A 1920 *times* 1080 30-frames/s 2.3 tops/w stereo-depth processor for energy-efficient autonomous navigation of micro aerial vehicles. *IEEE Journal of Solid-State Circuits*, 2017.
[2] H. An, S. Schiferl, S. Venkatesan, T. Wesley, and D. Sylvester. An ultra-low-power image signal processor for hierarchical image recognition with deep neural networks. *IEEE Journal of Solid-State Circuits*, PP(99):1–1, 2020.
[3] Shihui Yin, Zhewei Jiang, Jae-Sun Seo, and Mingoo Seok. Xnor-sram: In-memory computing sram macro for binary/ternary deep neural networks. *IEEE Journal of Solid-State Circuits*, 55(6):1733–1743, 2020.
[4] Win-San Khwa, Jia-Jing Chen, Jia-Fang Li, Xin Si, En-Yu Yang, Xiaoyu Sun, Rui Liu, Pai-Yu Chen, Qiang Li, Shimeng Yu, and Meng-Fan Chang. A 65nm 4kb algorithm-dependent computing-in-memory sram unit-macro with 2.3ns and 55.8tops/w fully parallel product-sum operation for binary dnn edge processors. In *2018 IEEE International Solid - State Circuits Conference - (ISSCC)*, pages 496–498, 2018.
[5] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012.
[6] Rui Liu, Xiaochen Peng, Xiaoyu Sun, Win-San Khwa, Xin Si, Jia-Jing Chen, Jia-Fang Li, Meng-Fan Chang, and Shimeng Yu. Parallelizing sram arrays with customized bit-cell for binary neural networks. In *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*, pages 1–6, 2018.
[7] Hyunjoon Kim, Qian Chen, and Bongjin Kim. A 16k sram-based mixed-signal in-memory computing macro featuring voltage-mode accumulator and row-by-row adc. In *2019 IEEE Asian Solid-State Circuits Conference (A-SSCC)*, pages 35–36, 2019.
[8] Mohsen Imani, Saikishan Pampana, Saransh Gupta, Minxuan Zhou, Yeseong Kim, and Tajana Rosing. Dual: Acceleration of clustering algorithms using digital-based processing in-memory. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 356–371, 2020.